COLLEGE / VLG PROJECT 2025-26

# Trendora

Personalized Smart Shopping website
inspired by Meesho, Myntra and Flipkart

Project report - PDF copy for submission

Department of Computer Science / Information Technology

# Contents

1. Certificate
2. Declaration
3. Acknowledgement
4. Abstract
5. 1. Introduction
6. 2. Literature survey
7. 3. System analysis and SRS
8. 4. System design
9. 5. Implementation
10. 6. Personalized Smart Shopping
11. 7. Testing
12. 8. Results and conclusion
13. 9. Future scope
14. 10. References
15. Appendix A - How to run
16. Appendix B - User manuals
17. Appendix C - Module list

# Certificate

This is to certify that the project titled "Trendora - Personalized Smart Shopping Website" is a bona fide record of work carried out as part of the academic curriculum. The software implements a Myntra-style customer storefront, a private admin desk, two-step OTP for shoppers, and a Personalized Smart Shopping layer (AI style assistant, mood, budget lock, complete-the-look, photo similar search, eco score, smart cart and Student Mode). The candidate has presented the working system, test cases and this report for evaluation.

# Declaration

I declare that this project is my original work. Public storefronts of Myntra, Flipkart and Meesho were studied only as user-experience references. Brand name Trendora, source code, catalogue copy and documents are original. No live payment gateway is connected. Simulated credentials are published for laboratory demonstration only. Public users are customers only; catalogue add/delete is reserved for the store admin.

# Acknowledgement

I thank my project guide, the Department of Computer Science / Information Technology, laboratory staff and classmates who reviewed the shopping flow and the Style Assistant. I also thank open documentation of React, Vite and React Router.

# Abstract

Indian e-commerce is defined by fashion discovery (Myntra), deals (Flipkart) and affordable catalogue (Meesho). Trendora unifies those habits in one React single-page application and then goes further: the focus is Personalized Smart Shopping, not only a bag.

Shoppers browse eight categories, search and filter, read size charts, check PIN codes, compare SKUs, use wishlist, coupons, gift-wrap, simulated UPI/card/COD checkout, order tracking, returns, Insider points and help tickets. New accounts use a two-step OTP (demo code on screen or 123456). The public site cannot add or delete products. The owner/admin enters a hidden /staff door to add items, hide SKUs, move orders and process returns.

The Smart layer (route /style) builds a complete outfit from a natural-language prompt, mood tiles, or a budget lock; product pages complete the look; the bag suggests missing pieces; photo upload finds similar colour tones on-device; every card can show Why we recommend this and an Eco Score; Student Mode surfaces campus prices and coupon CAMPUS10.

State lives in StoreContext and localStorage (key trendora-store-v2) so a viva survives refresh without a paid back-end.

# 1. Introduction

## 1.1 Motivation

College students already shop on Meesho, Myntra and Flipkart. A clone that only lists products is easy to dismiss. A system that styles the shopper - complete looks under a budget, mood, student mode - is easier to defend in a viva and closer to how people actually decide.

## 1.2 Problem statement

Design a responsive shopping website that covers the Indian shopping journey and a private admin desk, plus Personalized Smart Shopping, without a live warehouse or payment gateway. Public users must remain customers; only the site owner may change the catalogue.

## 1.3 Objectives

1. Study IA of Myntra, Meesho and Flipkart.
2. Implement customer shop: catalogue, search, filter, PDP, size chart, PIN, reviews, Q&A, wishlist, compare, bag, coupons, checkout, orders, cancel, return, Insider, studio, help.
3. Keep public login customer-only with 2-step OTP.
4. Provide a private admin desk at /staff for add / edit / hide products and order pipeline.
5. Implement Personalized Smart Shopping (nine modules listed in Chapter 6).
6. Persist data locally and ship PDF / Word / PPT documents.

## 1.4 Scope

In scope: modules above, INR pricing, simulated payments, college documents.

Out of scope: real Razorpay capture, courier AWB, native apps, cloud vision APIs.

## 2. Literature survey

Myntra - fashion PDP, size chart, Insider loyalty, Studio lookbooks, easy returns.

Flipkart - deal timer, coupon engine, MRP break-up, COD, order timeline.

Meesho - supplier listing, commission, category-first mobile bag.

Stitch Fix / Amazon Outfit - complete-the-look and budget outfits as personalization references.

Gap: a student cannot reimplement logistics or a large ML cluster. Trendora copies the nouns (bag, wishlist, COD, look) and implements them as original React code with an honest simulation note.

# 3. System analysis and SRS

3.1 Actors

Customer (public) - shop, OTP login/register, bag, pay, track, return, review, ticket. Cannot mutate catalogue.

Admin - private /staff login. Catalogue add/edit/hide, order pipeline, returns, coupons, tickets.

Owner - superset of admin plus people, roles, store policy, GMV reports.

3.2 Functional requirements

FR1 Multi-category catalogue with stock and variants.

FR2 Search / filter / sort (price, rating, discount, eco, mood, student).

FR3 PDP variants, size chart, PIN check, reviews, Q&A.

FR4 Bag, wishlist, compare (max 3).

FR5 Coupon engine (TREND10, FESTIVE20, WELCOME100, FREESHIP, INSIDER15, CAMPUS10).

FR6 Checkout with address sanitise, gift wrap, UPI/card/COD simulated.

FR7 Order timeline, cancel before ship, return after deliver.

FR8 Customer-only public auth + OTP (5 min, 5 tries, backup 123456).

FR9 Private staff door /staff; public /login rejects staff accounts.

FR10 Admin guided add-item (3 steps) and hide SKU.

FR11 Style prompt -> complete look under budget.

FR12 Mood shopping (7 moods).

FR13 Budget lock persisted.

FR14 Complete this look on PDP.

FR15 Photo similar search (dominant colour, on-device).

FR16 Why we recommend this + Eco Score + eco filter.

FR17 Smart cart missing-slot suggestion.

FR18 Student Mode + CAMPUS10.

FR19 Help tickets and notifications.

3.3 Non-functional

Responsive, INR format, no crash on empty bag, honest security disclaimer, lab-machine friendly.

# 4. System design

4.1 Architecture

Browser -> React pages -> StoreContext (use cases) -> products.js + styleEngine.js + localStorage.

Three layers: presentation, application state, persistence.

4.2 Routing

Public: /, /shop, /style, /product/:id, /offers, /brands, /studio, /insider, /compare, /help, /login, /register, /documents.

Customer: /cart, /checkout, /orders, /returns, /addresses, /wishlist, /profile, /notifications.

Hidden staff: /staff.

Guarded: /admin/*, /owner/* via RequireRole. Customers hitting /admin are sent home.

4.3 Price algorithm

grand = max(0, subtotal - coupon + shipping + giftWrap)

shipping = 0 if subtotal >= freeShipMin else shipFee (default 999 / 79).

giftWrap = 49 if selected.

Insider points += floor(grand / 10).

4.4 Order state machine

Confirmed -> Packed -> Shipped -> Out for delivery -> Delivered -> (optional) Returned.

Cancel allowed before Shipped. Return allowed only after Delivered.

4.5 Style engine

src/lib/styleEngine.js parses a Hindi/English prompt for budget, mood, college, gender. It greedy-fills slots top, bottom, shoes, bag, accessory under the remaining rupees. Photo search averages RGB on a canvas and nearest-neighbour matches catalogue colours.

4.6 Demo accounts

Customer: demo@trendora.in / demo123 then OTP on screen or 123456.

Admin: /staff then admin@trendora.in / admin123.

Owner: /staff then owner@trendora.in / owner123.

# 5. Implementation

Environment: Node.js 18+, npm, Vite 5, React 18, React Router 6, Context API, CSS design tokens.

Key modules:

src/context/StoreContext.jsx - auth, bag, orders, catalogue gates, studentMode, budgetLock, mood.

src/data/products.js - seed catalogue including campus SKUs, coupons, size charts, PIN helper.

src/lib/styleEngine.js - looks, mood, eco, why, photo colour, smart cart.

src/lib/security.js - SHA-256 passwords, OTP issue/check, login lockout.

src/pages/Style.jsx - Smart Shopping hub.

src/pages/dash/AddItem.jsx - guided 3-step add (admin/owner only).

src/pages/StaffLogin.jsx - private door.

src/components/RequireRole.jsx - route guard.

Security honesty: demo hashes live in localStorage. Production would hash on a server (bcrypt) and issue HTTP-only cookies. Stated in FAQ and viva notes.

# 6. Personalized Smart Shopping

This chapter is the project differentiator.

### 6.1 AI Style Assistant

User types: "???? college ?? ??? Rs 1000 ?? ???? outfit ??????" Engine extracts budget 1000, college=true, mood=casual and returns a complete look (top, bottom if needed, shoes, bag, accessory) with running total and leftover.

### 6.2 Mood shopping

Seven moods: Cute, Aesthetic, Minimal, Party, Casual, Formal, Traditional. Each product has inferred moods from name, tags and description.

### 6.3 Budget lock

Persisted integer. Looks and shop max-price respect it. Example: Complete look Rs 1299 inside a Rs 1500 lock.

### 6.4 Complete this look

On PDP, complementary slots are suggested (earrings, footwear, bag, watch). One-piece dresses skip a bottom.

### 6.5 Photo similar search

File stays in the browser. Canvas 40x40 average colour -> nearest named tone -> ranked SKUs. No cloud API, suitable for a college lab.

### 6.6 Why we recommend this

Bullets: college friendly, in budget, mood match, good ratings, eco pick, bestseller.

### 6.7 Eco Score

0-10 from materials (organic, linen, handcrafted raise; electronics lower). Shop filter Eco 8+.

### 6.8 Smart cart

After add-to-bag, missing outfit slots trigger an optional suggestion ("Matching earrings sirf Rs 199 mein add karein?").

### 6.9 Student Mode

Toggle on /style or shop filters. Surfaces campus SKUs (crop, jeans, tote, kurti, polo, notebook, pens, sleeve,

flips) and unlocks CAMPUS10 (10% above Rs 499).

# 7. Testing

T1 Home hero + categories render - Pass

T2 Search earbuds finds PulseBuds - Pass

T3 Filter Women + sort price - Pass

T4 Style prompt college under 1000 returns a look whose total <= 1000 - Pass

T5 Mood Party changes look composition - Pass

T6 Budget lock 1500 hides over-budget look total - Pass

T7 PDP Complete this look shows other slots - Pass

T8 Photo upload lists similar products - Pass

T9 Eco filter 8+ reduces catalogue - Pass

T10 Smart cart suggests missing accessory - Pass

T11 Student Mode + CAMPUS10 applies only when mode is on - Pass

T12 Public /login rejects admin@trendora.in - Pass

T13 /staff admin can publish a new SKU; it appears on Shop - Pass

T14 Customer OTP accepts on-screen code or 123456 - Pass

T15 FESTIVE20 rejected below Rs 1999 - Pass

T16 Place order empties bag and drops stock - Pass

T17 Admin pipeline Packed -> Delivered; customer can request return - Pass

T18 /admin as customer redirects home - Pass

T19 Refresh keeps session (trendora-store-v2) - Pass

# 8. Results and conclusion

Trendora presents a credible Indian storefront plus a Smart Shopping layer that a viva panel can click through in under five minutes: type a college prompt, lock a budget, open a dress, see complete-the-look, add to bag, accept a smart suggestion, checkout (simulated).

Public users never see add/delete. Admin work stays on /staff. Context + localStorage is a pedagogical stand-in for REST; swapping placeOrder for fetch('/api/orders') is a weekend of work, not a rewrite.

Conclusion: the project meets its objectives. It is demoable, documented (PDF, Word, PPT) and honest about simulation.

# 9. Future scope

1. Express + MongoDB API.
2. Razorpay / Stripe test mode.
3. Real SMS/email OTP.
4. Cloud vision for photo search.
5. Learned size recommender.
6. PWA offline catalogue.
7. Jest + React Testing Library.

# 10. References

1. React documentation - https://react.dev
2. Vite guide - https://vitejs.dev
3. React Router - https://reactrouter.com
4. Nielsen Norman Group, e-commerce UX heuristics
5. Public storefronts of Myntra, Flipkart and Meesho (UX reference only)
6. MDN Web Docs - Web Storage API, Canvas API
7. Stitch Fix / Amazon Outfit pages (personalization reference only)

# Appendix A - How to run

cd new-shopping-website

npm install

npm run dev

Open the printed URL (default http://localhost:5173).

Production: npm run build && npm run preview.

Regenerate these documents: .venv-docs/bin/python scripts/generate_docs.py

# Appendix B - User manuals

Customer: /login demo@trendora.in / demo123 -> OTP -> shop or /style -> bag -> CAMPUS10 (if Student Mode) or FESTIVE20 -> checkout. Payments are fake.

Admin (site owner only): footer Staff or type /staff -> admin@trendora.in / admin123 -> Add a new item (3 steps) -> Publish -> open Shop. Catalogue Edit / Hide. Orders pipeline.

Style Assistant: Nav Style -> type a Hindi/English wish -> Suggest complete outfit -> Add full look to bag.

Photo search: Style -> Photo search tab -> choose image -> similar products.

# Appendix C - Module list

Home, Shop, Style Assistant, Product detail, Offers, Brands, Studio, Insider, Compare

Bag, Checkout, Orders, Returns, Addresses, Wishlist, Profile, Notifications, Help, FAQ, Documents

Login (customer OTP), Register (customer OTP), Staff login (hidden)

Admin: home, guided add, catalogue, orders, returns, coupons, tickets, users

Owner: all admin + reports + settings + create staff